

CS3103
Assignment 4
Group 7 Report

Yoon Jia Bao (A0307920J)
Soon Fu Hong Wilson (A0273596M)
Nicole Teh May Xin (A0342054N)
Chin Shang Zer Ian (A0307824A)

Table of Contents:

Architecture **3**
Protocol **3**
Revision/Recovery Semantics **4**
Backpressure/Overload Policy **4**
Public Experiments **5**
Test Environment 5
Optimisation Comparison **5**
Impact of Reset and Resume 6
Results **7**
Failures and Trade-offs **8**
Limitations **9**
Future-work/FYP direction **10**
References **11**

Architecture

LiveRecall has three components connected by WebSockets over TCP: the caption source, the server (solution_server.py) and the browser client (client/), as shown in Figure 1.

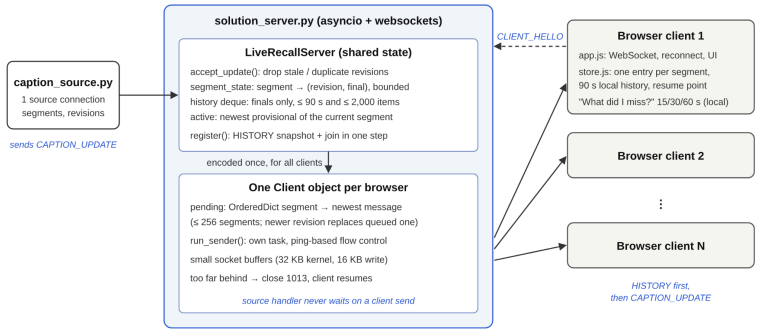


Figure 1: Overall architecture of the LiveRecall system.

Server. Built with Python asyncio and websockets. It rejects outdated or duplicate revisions, keeps finalized captions for up to 90 s or 2,000 items, and gives each browser its own queue, so a slow client cannot delay the others.

Browser client. store.js holds caption state and a local history of finalized captions; app.js handles the WebSocket connection, automatic reconnection and display. “What did I miss?” shows the last 15, 30 or 60 s from this local history.

Protocol

Peers exchange JSON messages over WebSockets (TCP). Every connection must start with SOURCE_HELLO or CLIENT_HELLO; otherwise, or if no hello arrives within 5 s, the server closes it.

Table 1: Main protocol messages.

Message	Direction	Purpose
CAPTION_UPDATE	Source - Server - Clients	Send new or updated captions.
CLIENT_HELLO	Clients - Server	Identifies the client and requests caption history.
HISTORY	Server - Client	Sends recent finalized captions.

Connection sequence. When a browser client connects, it sends a CLIENT_HELLO message containing its client ID, session information, and latest finalized caption position (0 for a new client). The server then retrieves the relevant caption history and registers the client in one step

without an asynchronous wait, preventing missed or duplicated updates between history retrieval and live streaming. The server sends a HISTORY message containing recent finalized captions, followed by the latest provisional caption if one exists. After that, the client receives live CAPTION_UPDATE messages. The browser displays “Syncing...” while receiving history and changes to “Live” once synchronisation is complete. WebSocket ping messages are used by the server to monitor client responsiveness.

Revision/Recovery Semantics

Each caption is identified by session, segment number, revision, and finality. The server rejects stale or duplicate revisions using metadata for up to 4,096 segments and a finalised-segment watermark. The browser stores one caption per segment, replacing older provisional revisions while preserving finals. Its state is pruned to 90 seconds and 2,000 entries, supporting 15-, 30-, or 60-second recall.

On disconnection, the browser reconnects with exponential backoff (0.5–5 seconds). CLIENT_HELLO supplies the latest finalised cursor and requests up to 60 seconds of history. The server retains finals for 90 seconds or 2,000 entries and returns newer retained finals through HISTORY. Atomic snapshotting and registration prevent recovery gaps, while shared revision rules prevent duplicates. Source takeover preserves state but requires continuous segment numbering.

Recovery restores retained finals, not missed provisional revisions. Browser reloads and server restarts clear their respective states, so recovery is not guaranteed beyond retention limits. Across three T4 runs, final completeness remained at 100%, while recovery P95 improved from 4.51 ms to 2.72 ms, measured from connection establishment to HISTORY receipt.

Backpressure/Overload Policy

Each client has an independent sender task and a 256-entry pending queue, isolating slow readers. Newer provisional revisions replace queued versions, while finals are preserved. Shared history remains separate from client queues.

Flow-control pings are sent every four messages. Slow readers are detected through two consecutive waits exceeding max(1 s, 4 × RTT baseline) or an unanswered ping exceeding max(4 s, 8 × RTT baseline). Detected clients enter finals-only mode and may be reset using close code 1013. Reconnection recovers missed finals through HISTORY, with an initial 60-second finals-only period and a one-minute reset cooldown. Reset tracking is bounded to 10,000 client IDs over five minutes. The --no-reset-slow option disables proactive resets.

A 32 KiB socket send buffer, 16 KiB WebSocket write high-water mark, and disabled compression support backpressure and reduce CPU overhead, although downstream buffering remains outside application control.

Across three T3 runs, final completeness improved from 98.42% to 100%, while the Fresh Final Caption Ratio increased from 90.00% to 96.58%. Overall P95 caption age fell from 1,827.39 ms to 4.34 ms. Normal-client P95 improved to 3.97 ms, while slow-client P95 remained at 9,614.55 ms. Ten intentional reset-related connection errors were recorded. T4 and T5 maintained 100% final completeness with no connection errors in the reported median results, highlighting the trade-off between reliable recovery, timely delivery, and connection stability.

Public Experiments

We ran all five public tests from PUBLIC_TESTS.md. Each test was run three times with run_demo.py --runs 3, which restarts the server for each run and we report the median.

Test	Command arguments	What it checks
T1 - Basic correctness	--clients 20 --duration 20	Completeness, no duplicates or stale state, revision coverage
T2 - Public scale	--clients 100 --duration 40	Caption-age tail and freshness at 100 clients
T3 - Slow clients	--clients 100 --duration 60 --slow-percent 10 --slow-ms 500	Whether slow clients hurt normal ones and memory stays bounded
T4 - Reconnect	--clients 100 --duration 60 --disconnect-percent 20 --disconnect-at 15 --disconnect-seconds 5	HISTORY recovery time and completeness after a drop
T5 - Network impairment	T2 under network_profiles/delay_loss.sh lo	Tail latency under delay and loss and no false slow-client detections

Test Environment

All tests ran on Intel(R) Core(TM) i7-14700KF (28 cores), 64GB RAM, Windows 11 + WSL2 Ubuntu 24.04.01, Python 3.12.3 and websockets 16.1.1. The server, load clients and caption source ran on the same machine over loopback, so the caption age measures queueing and processing delay, not real network delay. T5 used tc/netem on the loopback interface via network_profiles/delay_loss.sh lo, which adds 120 $\pm$ 30ms delay and 3% loss.

Optimisation Comparison

Both servers are measured with the same commands on the same machine, with the median of three runs. The initial server was the version before finals-only mode for slow clients was implemented. The final server increased T3 final completeness from 98.42% to 100%. P99 also fell 42% from 9261.99ms to 5401.99ms. Normal clients stayed just as fast or slightly faster in the final server. T5 varies little between the two server versions, because the delay and loss comes from the network instead of the servers. In that case, we optimise the server to not trigger slow client logic on healthy clients accidentally due to network circumstances.

Additionally, a slow client takes about 8s in this case to be detected, because its own WebSocket library has to fill its 16-message buffer first. Captions in that window arrive late and make up the slowest 5%. After the reset, the client stays up-to-date.

Metric (median of 3)	T3 initial	T3 final	T4 initial	T4 final	T5 initial	T5 final
Final completeness	98.42%	100%	100%	100%	100%	100%
Fresh Final Caption Ratio	90.00%	96.58%	98.42%	98.61%	89.22%	90.69%
Fresh Caption Ratio	94.74%	98.27%	100%	100%	92.84%	93.34%
P50 caption age	2.73 ms	2.09 ms	2.68 ms	2.62 ms	252.68 ms	250.07 ms
P95 caption age	1827.39 ms	4.34 ms	5.41 ms	4.81 ms	611.35 ms	602.31 ms
P99 caption age	9261.99 ms	5401.99 ms	7.30 ms	5.99 ms	1087.15 ms	1008.52 ms
Normal-client P95	5.47 ms	3.97 ms	5.41 ms	4.81 ms	611.35 ms	602.31 ms
Slow-client P95	9747.24 ms	9614.55 ms	-	-	-	-
Recovered HISTORY items	0	130	130	130	-	-
Recovery P95	-	1.36 ms	4.51 ms	2.72 ms	-	-
Connection errors	0	10	0	0	0	0

Optimisation	Problem in our initial server before further optimisation	Measured effect
Finals-only mode for slow clients	Slow clients spent their bandwidth on old provisionals	T3 completeness increased from 98.4% to 98.95%, but each slow clients still trailed by about 6s, which led to the next optimisation
Reset and resume through HISTORY	Slow clients might not read fast enough resulting in stale messages in their WebSocket library	T3 completeness increased from 98.95% to 100%
Compression off	Each ~200-byte caption was compressed per client	Server CPU time fell 38% (4.9s to 3.0s for a 30s run with 500 clients, on a separate test), and P95 went from 70 ms to 63 ms (median). The cost is about twice more bytes per message

Impact of Reset and Resume

Tested on T3 on the same final server, but different machine from previously.

Metric (median of 3)	Reset off	Reset on (final)
Final completeness	99.08%	100%
Fresh Final Caption Ratio	90.00%	96.58%
Slow-client median caption age	6.3s	11ms
Slow-client fresh ratio	1.7%	58%
P99 caption age	8355ms	4375ms
Normal-client P95	17.15ms	14.30ms
Connection errors	0	10
Normal Update Coverage	95.18%	93.90%

The reset handles slow-client's lag. With the reset off, flow control and finals-only mode still bring completeness to 99.08%, but each slow client stays about 6s behind. Those seconds are the approximately 16 messages queued in its own WebSocket library, where the server cannot access them. Closing the connection discards that queue and one HISTORY message delivers every final caption the client missed.

Results

The final server reaches 100% completeness on all five public tests. On T1 to T4, normal-client P95 caption age stays within 5.23ms. Also, no run had a duplicate or stale message. Each value is the median of 3 independent runs of the final server run on the same machine.

Metric	T1	T2	T3	T4	T5
P50 caption age	0.93ms	3.16ms	2.09ms	2.62ms	250.07ms
P95 caption age	1.50ms	5.23ms	4.34ms	4.81ms	602.31ms
P99 caption age	1.94ms	6.14ms	5401.99ms	5.99ms	1008.52ms
Fresh Caption Ratio	100%	100%	98.27%	100%	93.34%
Fresh Final Caption Ratio	100%	100%	96.58%	98.61%	90.69%
Final completeness	100%	100%	100%	100%	100%
Normal Update Coverage	100%	100%	93.90%	98.42%	97.94%
Normal-client P95	1.50ms	5.23ms	3.97ms	4.81ms	602.31ms
Recovery P95	-	-	1.36ms	2.72ms	-
Duplicate / Stale messages	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0

Connection errors	0	0	10	0	0

- For T1/T2, 99% of captions arrived within 6.14 ms.
- For T3, the 5.4s P99 comes only from the 10% of slow clients. Normal clients stay at 3.97ms P95, so a slow client does not slow down others. The 10 connection errors are intentional resets that give 100% completeness. During one T3 run, server memory (RSS, sampled every 1 s with monitor_server.py) stayed between 27.56 MB and 27.78 MB and was flat for the last part of the run. The server's status log, sampled every 10 s, showed a largest per-client backlog of at most 1 of the 256-entry cap, and 76 retained history items, under the 2,000 limit. So the 10 slow clients did not cause memory or queue growth.
- For T4, each dropped client recovers its missed finals in one HISTORY message. Fresh Final is below 100% as captions sent during the 5s outage arrive late through HISTORY.
- For T5, the 250ms P50 is added link delay and TCP retransmission stretches the tail to a P99 of 1s. Freshness falls as that tail crosses 500ms. However, we achieved 100% completeness and no client was falsely flagged as slow.

Failures and Trade-offs

The first server does not deliver every final caption to slow clients, even though two safeguards are in place. Firstly, it keeps only the newest version of each caption in a client's queue. Secondly, when a client leaves a ping unanswered for over 1s, the server sends it only final captions. The problem is that the second protection switches off as soon as the ping is answered, even though the client is still behind. The server then sends it draft captions again, and the client falls further behind.

To increase final completeness rate, slow clients are kept on finals only for 10s. This raises completeness to 98.95% but each slow client still lags by about 6s, delay being from the client's WebSocket library which holds up to 16 messages before the client reads them where the server cannot change or take back those messages. Only reset-and-resume reaches 100% final completeness by closing the connection to clear held messages and then sending every missed final in one HISTORY message.

The table shows the trade-offs between all the design choices implemented in our project:

Design choice	**Benefit**	**Cost**
Keep only the newest version of each caption per client	Each client's queue is capped at 256 captions. Slow clients skip outdated drafts	Slow clients see fewer draft updates
Reset-and-resume	T3 completeness rises from 99.08% to 100% (reset off vs on)	Planned disconnect per slow client (10 connection errors in T3). Slow clients get only finals for 60s after reconnecting, so T3 normal update coverage falls from 95.18% to

		93.90%
Slow client detection (ping unanswered for 4s, or two waits over 1s in a row)	Healthy clients on a slow and lossy network (T5) are rarely reset (1 out of 3 runs on a second machine reset one client).	A slow client is detected only after ~8seconds
Captions are sent at full size without shrinking each message before sending	Compressing costs CPU time for captions sent to every client. With 500 clients, that adds up and turning it off cuts server CPU time by 38% (500 clients, 30s run).	Each message is twice the bytes compared to shrinking
Small send buffers	Compared to bigger send buffers, less data is stuck below the app, so more drafts can still be coalesced and replaced.	Speed per connection is limited, which is fine for ~200-byte captions.
Asyncio event loop	A new client gets the caption history and joins the client list in one step and no lock, guaranteed to receive full history up to 60s.	Server is limited to using only one CPU core

We accept the cost of one connection error per reset because it keeps each client's queue bounded and gives 100% completeness, and we allow at most one reset per client per minute. T3 Normal Update Coverage, the share of all source revisions clients receive, falls to 93.90% because a slow client gets only finals for 60s after a reset.

Normal clients still get every update while detecting slow clients means choosing between speed and accuracy.

Limitations

While working on the project, we also encountered constraints that are fundamentally limiting to the system we built and couldn't fix. These limitations include:

- **Slow clients are detected after they fall behind.** Backlog only starts being observable to the server when the ~16-message client-side buffer fills and as a result, captions sent in the first ~8 s to a client will arrive late and account for the slowest captions in T3 tail (P99 5.4 s, slow-client P95 9.6 s).
- **The server cannot distinguish whether captions are shown on screen.** A ping reply only proves that the client's device received the data, not that the app displayed it. For example, a frozen browser tab still answers pings, and the server cannot distinguish its actual state.
- **Recovery is bounded**. The server keeps final captions in a history queue and each new final caption joins the back of the queue. Captions older than 90s or past the 2,000-caption limit, drop off the front. When a client reconnects, HISTORY sends it at most the last 60s of this queue. A client that is away for longer, and one that refreshes the page misses every caption sent before that and loses its own saved captions

respectively, and they are only able to receive only the last 60s. If the server restarts, the queue is emptied, so a reconnecting client gets nothing back.

- **One lecture session per server, and no security.** A new lecture needs a server restart. There is no login authentication or encryption (TLS). To avoid losing captions when the lecturer reconnects, a new source replaces the old one at once. With no login, anyone who sends SOURCE_HELLO for the session can take over as the source.
- **Evaluation conditions.** All tests run on one machine, so caption age measures time spent queuing and processing inside the server, not time on the wide-area network across a real network.

Future-work/FYP direction

In this project, we only use TCP, the transport WebSocket runs on. TCP has 2 weaknesses:

1. TCP delivers everything in strict order. When one packet is lost, every message behind it waits until that packet is resent. This is what stretches the T5 tail to a P99 of 1s.
2. Once a draft is sent, the server cannot take it back. Outdated drafts sit in the client's buffer, and slow clients spend time reading captions that are already old.

A future-work or FYP direction could look at rebuilding LiveRecall's transport on QUIC [1] through WebTransport in the browser, and treat drafts and finals differently:

1. Drafts can be treated as unreliable datagrams to be sent [2]. A lost draft is simply not retransmitted, because a newer draft or the final replaces it anyway.
2. Each final is sent on its own reliable QUIC stream. QUIC streams are independent, so a lost packet only delays its own caption and not every caption behind it, directly resolving a TCP weakness. This targets the T5 tail directly.
3. Superseded drafts can be cancelled in flight. QUIC allows the sender to reset a stream [1], so when a newer revision of a caption is ready, the server can cancel the older one that is still in transit. This addresses the client-buffer problem that forces us to reset slow clients today.

Success metrics to determine whether the approach makes the system more efficient: Rerun T3 and T5 over QUIC under the same delay and loss. The FYP direction succeeds if the T5 P99 falls well below 1s, slow clients need fewer resets, and completeness stays at 100%.

References

[1] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, 2021.
[2] T. Pauly, E. Kinnear and D. Schinazi, "An Unreliable Datagram Extension to QUIC," RFC 9221, IETF, 2022.